Segurança da Informação– GBC083

Prof. Rodrigo Sanches Miani – FACOM/UFU

Tópicos

1. Resumo da semana 10
2. Direcionamentos para a semana 11
3. Dúvidas??



Resumo da semana 11

Segurança da Informação– GBC083

Resumo da aula...

- RSA
  - Algoritmo que implementa os requisitos de criptografia de chave pública;
  - Usa aritmética modular, primalidade e outros argumentos da **teoria de números**;
  - A descrição é razoavelmente simples mas a escolha dos parâmetros pode dificultar a sua implementação (e levar a escolhas muito ruins!);
  - A segurança do algoritmo está diretamente relacionada ao problema de fatoração de números inteiros.



Descrição do algoritmo – Geração de chaves

Geração de chave por Alice	
Selecione p, q	p e q são primos, $p \neq q$
Calcule $n = p \times q$	
Calcule $\phi(n) = (p - 1)(q - 1)$	
Selecione o inteiro e	$\text{mdc}(\phi(n), e) = 1; 1 < e < \phi(n)$
Calcule d	$d \equiv e^{-1} \pmod{\phi(n)}$
Chave pública	$PU = \{e, n\}$
Chave privada	$PR = \{d, n\}$



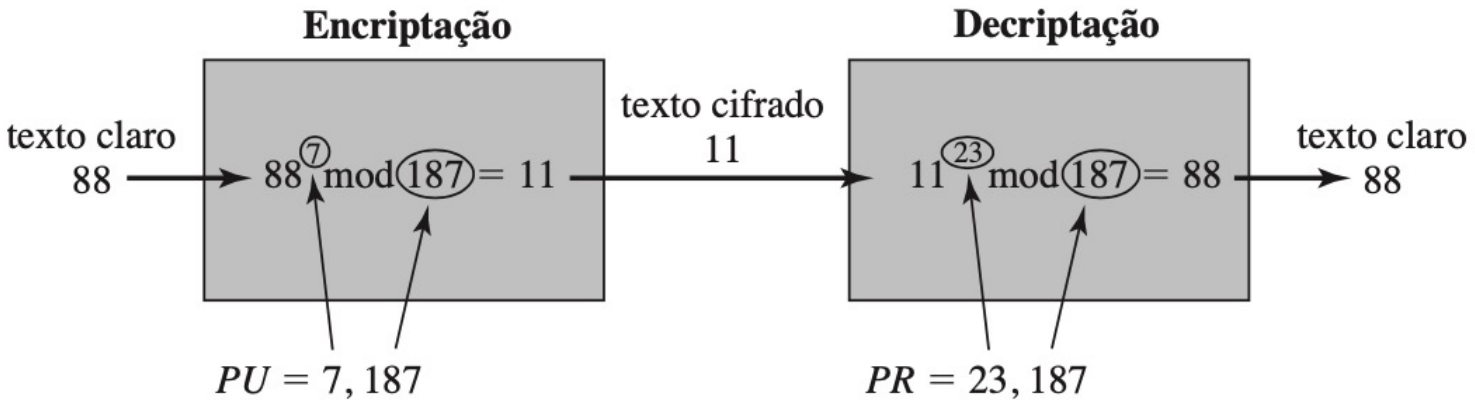
Descrição do algoritmo – Cifrar/Decifrar

Encriptação por Bob com chave pública de Alice	
Texto claro:	$M < n$
Texto cifrado:	$C = M^e \bmod n$

Decriptação por Alice com a chave privada de Alice	
Texto cifrado:	C
Texto claro:	$M = C^d \bmod n$



RSA - Exemplo


Encriptação
Decriptação
texto claro
88
$88^{7} \bmod 187 = 11$
texto cifrado
11
$11^{23} \bmod 187 = 88$
texto claro
88
$PU = 7, 187$
$PR = 23, 187$

Direcionamentos para a semana 11

Segurança da Informação– GBC083

Direcionamentos para a semana 11

- Um conjunto de slides
  1. Tópico 11 – Funções de hash

- Uma videoaula:
  1. Tópico 11 – Funções de hash (52 minutos)



Direcionamentos para a semana 11

- QP7 – 23/02
  - Criptografia assimétrica e RSA
- TP5 disponível em breve – penúltimo!
- Como foi o TP-4? Esclareceu os conceitos teóricos? Brigaram muito com o OpenSSL?
- Semana que vem não teremos aula! Recesso de carnaval – de acordo com o calendário acadêmico.
  - http://www.prograd.ufu.br/sites/prograd.ufu.br/files/media/arquivo/resolucao_n._25-2020_-_congrad.pdf



Tópicos da aula – Funções de hash (videoaulas)

1. Ideia
2. Exemplo simples
3. Função de hash criptográfica
4. Ataques
5. Exemplos
6. Aplicações



Resumo da aula...

- Vocês já sabem:
  - Principais modelos de criptografia (simétrico e assimétrico);
  - Funcionamento dos principais algoritmos de criptografia (AES e RSA);
  - Como modelos criptográficos são implementados na prática (modelo híbrido);
  - Ou seja, a confidencialidade (sigilo) das comunicações estão razoalvelmente cobertas;
  - O que falta??



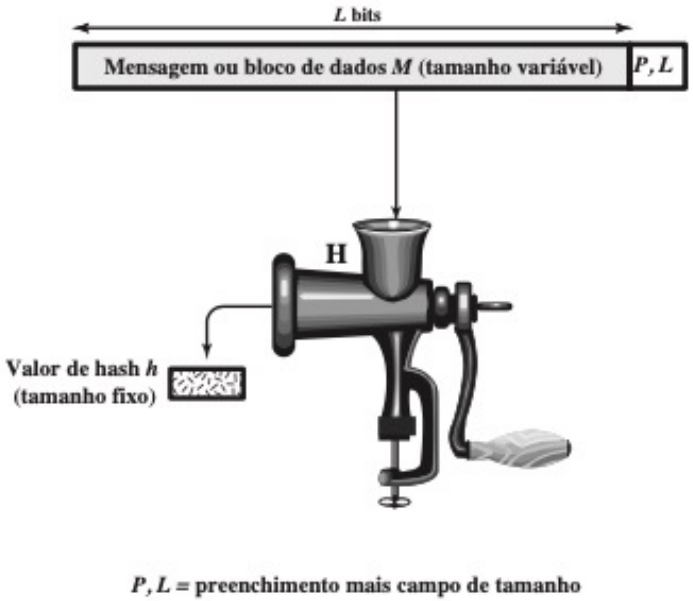
Resumo da aula...

- Vocês já sabem:
  - Principais modelos de criptografia (simétrico e assimétrico);
  - Funcionamento dos principais algoritmos de criptografia (AES e RSA);
  - Como modelos criptográficos são implementados na prática (modelo híbrido);
  - Ou seja, a confidencialidade (sigilo) das comunicações estão razoalvelmente cobertas;
  - O que falta??
  - Integridade! Autenticidade! Não-repúdio! Ou seja, a segurança não é composta somente de garantias de sigilo...



Ideia

- Uma **função hash** *H* recebe como argumento uma mensagem *M* de qualquer tamanho e retorna um valor de hash *h* de tamanho fixo:
  - $h = H(M)$
  - h também é conhecido como resumo de M



Função de Hash simples

- A entrada (mensagem, arquivo...) é vista como uma sequência de blocos de n bits;
- A entrada é processada um bloco de cada vez, para produzir uma função de hash de n bits;
- Uma das funções de hash mais simples é o XOR bit a bit de cada bloco:

$$C_i = b_{i1} \oplus b_{i2} \oplus \ldots \oplus b_{im}$$

onde

$C_i = i$-ésimo bit do código de hash, $1 \leq i \leq$ n

$m =$ número de blocos de n bits na entrada

$b_{ij} = i$-ésimo bit no j-ésimo bloco

$\oplus =$ operação XOR



Requisitos

- Uma função de **hash criptográfica forte** deve respeitar os seguintes requisitos:
  1. Aceitar mensagens de entrada de tamanho variável.
  2. Retornar valores de tamanho fixo.
  3. Deve ser eficiente em termos de complexidade computacional.



Requisitos

- Uma função **hash criptográfica forte** deve respeitar os seguintes requisitos:
  4. **Resistência à 1ª inversão (pré-imagem)**, ou seja, é computacionalmente inviável, dado y, encontrar x tal que h(x) = y;
  5. **Resistência à 2ª inversão (segunda pré-imagem)**, ou seja, é computacionalmente inviável, dado x1, encontrar x2 ≠ x1 tal que h(x1) = h(x2);
  6. **Resistência a colisões**, ou seja, é computacionalmente inviável, encontrar quaisquer x1 e x2 tais que h(x1) = h(x2).

- **Ataques** em funções de hash envolvem medir o custo envolvido para encontrar a pré-imagem, segunda pré-imagem e colisões

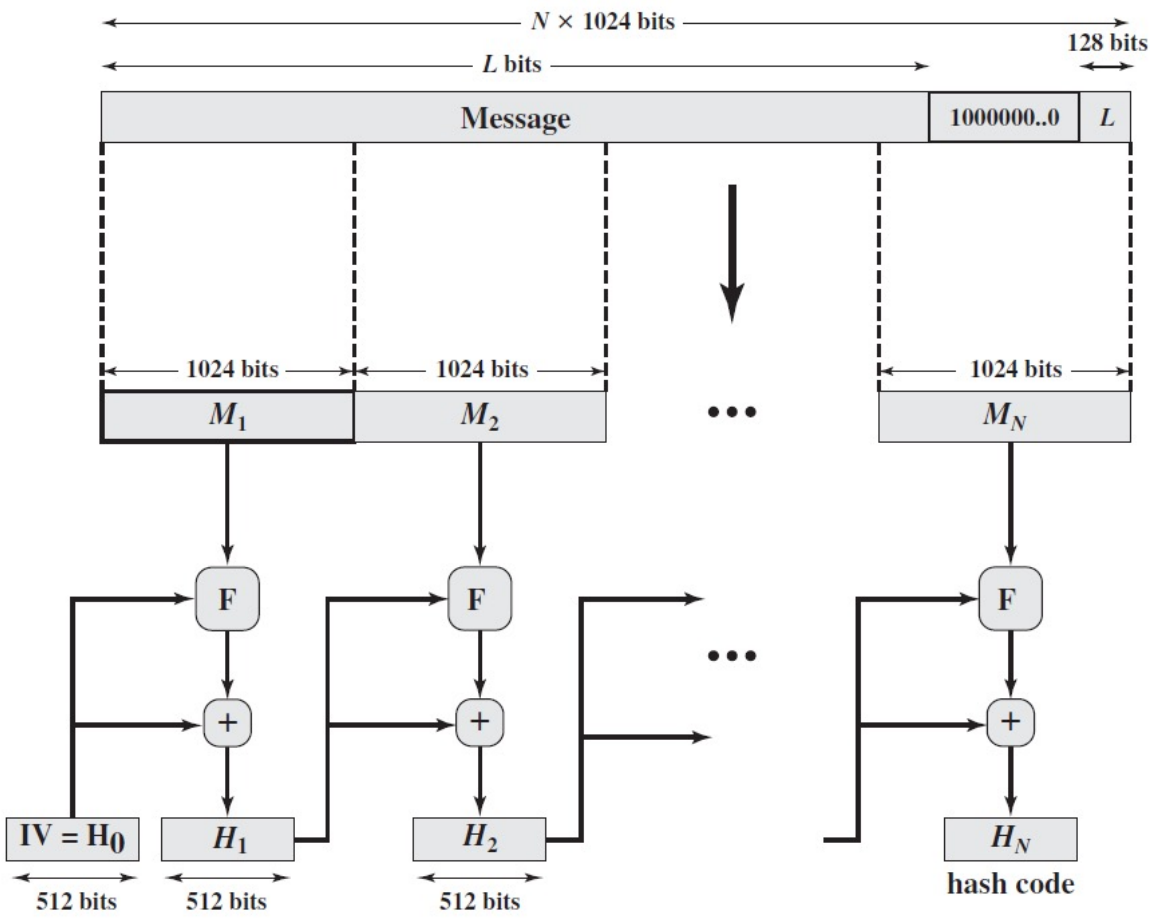


Exemplos

- As funções das famílias MD (Message digest) e SHA (Secure Hash) são bem famosas...
- MD5 é uma função problemática do ponto de vista de segurança e avisos sobre não usá-la em aplicações críticas são feitos desde 2011;
- No começo de 2017 pesquisadores encontraram falhas (colisões) no SHA1. O ataque foi implementado com o auxílio da infraestrutura computacional do Google;
- A recomendação atual é utilizar funções como SHA2 e SHA3.
- https://arstechnica.com/information-technology/2017/02/at-deaths-door-for-years-widely-used-sha1-function-is-now-dead/
- https://www.computerworld.com/article/3173616/security/the-sha1-hash-function-is-now-completely-unsafe.html



SHA 512 – Visão geral



1. Adição de bits de padding e do tamanho do bloco;
2. Inicialização de um buffer que armazenará os resultados;
3. Processamento dos n blocos (1024 bits).;
4. A função f (função de compressão) cada bloco passa por 80 rodadas de deslocamentos circulares, AND, OR, XOR;
5. A saída do estágio N é o hash da mensagem M.

https://github.com/in3rsha/sha256-animation

MD5 e SHA

```
(base) MacBook-Pro-de-Rodrigo-7:~ rodrigomiani$ echo teste | md5
1ca308df6cdb0a8bf40d59be2a17eac1
(base) MacBook-Pro-de-Rodrigo-7:~ rodrigomiani$ echo teste | shasum
9dc628289966d144c1a5fa20dd60b1ca1b9de6ed  -
```

```
(base) MacBook-Pro-de-Rodrigo-7:~ rodrigomiani$ echo testi | md5
e20752b108d3cb3dd47adb44e5535221
(base) MacBook-Pro-de-Rodrigo-7:~ rodrigomiani$ echo testi | shasum
30b278045442d5501937a0df06eb81d58de839e9  -
```

https://github.com/in3rsha/sha256-animation



Aplicações

1. **Verificação de integridade de arquivo;**
2. **Armazenamento de senhas;**
3. Autenticação de mensagem (MACs);
4. Assinatura digital.

3 e 4 serão assuntos das próximas aulas.



Verificação de integridade de arquivo

	Name	Last modified	Size	Description
	Parent Directory		-	
	FOOTER.html	2020-08-06 12:30	810	
	HEADER.html	2020-08-07 06:01	3.9K	
	SHA256SUMS	2020-08-06 12:30	202	
	SHA256SUMS.gpg	2020-08-06 12:30	833	
	ubuntu-20.04.1-desktop-amd64.iso	2020-07-31 13:52	2.6G	



Verificação de integridade de arquivo

	Name	Last modified	Size	Description
	Parent Directory		-	
	FOOTER.html	2020-08-06 12:30	810	
	HEADER.html	2020-08-07 06:01	3.9K	
	SHA256SUMS	2020-08-06 12:30	202	
	SHA256SUMS.gpg	2020-08-06 12:30	833	
	ubuntu-20.04.1-desktop-amd64.iso	2020-07-31 13:52	2.6G	

```
b45165ed3cd437b9ffad02a2aad22a4ddc69162470e2622982889ce5826f6e3d *ubuntu-20.04.1-desktop-amd64.iso
443511f6bf12402c12503733059269a2e10dec602916c0a75263e5d990f6bb93 *ubuntu-20.04.1-live-server-amd64.iso
```



Armazenamento de senhas



Dúvidas??

Segurança da Informação– GBC083